UTCP

Alex Taschuk

COSC 448: Directed Studies in Computer Science

Professor Scott Fazackerley

2026 Winter Term 2

Network reliability has a very non-deterministic behavior due to a wide array of influences, many of which are often not visible or in our control. Two significant factors that contribute to this behavior are the speed at which a peer can send/receive data over the network (hardware limitations) and the speed at which a peer's application can read/write data to the peer's send and receive buffer. Congestion control algorithms such as TCP Tahoe, Reno, and NewReno were developed to help prevent and manage network congestion. These algorithms are designed to work under a variety of network conditions in order to avoid and handle congestion at any given moment, and they rely on two main concepts. The first is called flow control, and it helps protect a peer's application from being sent too much data too quickly. The receiving peer advertises a receive window (rwnd) to inform the sending peer how much data their application can handle receiving. The second concept is called congestion control, and it helps protect a peer's network by advertising a congestion window (cwnd) to ensure that the sending peer does not send more data than the network can handle. Together, the rwnd and cwnd put a ceiling on how many unacknowledged (unACKed) bytes the sender can have in-flight at once. With this information, peers can minimize the likelihood of exceeding a network's and/or application's capacity to receive data.

Given this, there are two goals for this directed study. The first is to gain a deeper understanding of how TCP and its congestion control algorithms are implemented (specifically in C). The second is to explore the potential of using a long short-term memory (LSTM) neural network to detect anomalous network patterns. The idea behind the second goal is that if an LSTM can reliably identify anomalous patterns, a sender could dynamically adjust its congestion window based on the network's current reliability, rather than waiting for 3 duplicate acknowledgments (ACKs) or a timeout. For example, the LSTM might recognize that at one moment the slow start threshold (ssthresh) needs to drop only to 75% of the congestion window (cwnd), whereas at another moment it could be more effective to drop it to 25% of the cwnd[1].

[1] This use case is out of the scope of this research. We only wish to determine the viability of anomalous network pattern detection via an LSTM, but I thought the context might be helpful.

UTCP

Major operating systems implement transport-layer protocols at the kernel level[2], and safely modifying them is very complicated. A program can access the transport layer to manage connections, send data, and receive data through the Berkeley sockets API (or BSD sockets) via functions such as `connect()`, `send()`, and `recv()`. However, one goal of this research was to understand how the operating system (OS) implements TCP, and the best way to do this was to write our own OS-level implementation of TCP rather than use BSD socket functions to create a TCP client and server. Additionally, we needed to be in control of how an application sends and receives data over TCP at the transport layer. This approach has been dubbed "UTCP" by Sam. In essence, UTCP involves sending data between two peers over UDP. The payload of a UDP datagram contains a TCP segment, so it is technically demultiplexed (demuxed) twice: first to deliver the datagram to the correct UDP socket—this is handled by the kernel—then the TCP segment is demuxed to deliver the payload to the correct UTCP socket. Since packets are being sent over UDP and we are implementing TCP ourselves, we need a way to maintain the connection-oriented, reliable service that TCP promises. This means that all TCP-related logic is handled through UTCP rather than the kernel, giving us complete control over the protocol. From an application perspective, data is sent over UDP.

There are many parallels between standard kernel implementations of TCP and our simpler UTCP version. Every peer maintains a Transmission Control Block (TCB) for a connection that is either in the process of establishing a connection through a three-way handshake (3WHS) or has completed a 3WHS and is in an `ESTABLISHED` state. A TCB contains all the necessary information to maintain a connection between two peers and several key variables and structs necessary for the features of TCP that we have implemented. For instance, like TCP, a UTCP connection is identified by a four-tuple, which is made up of the source UTCP port, source IPv4 address, destination UTCP port, and destination IPv4 address.

[2] For example, as of version 2.6.19, the Linux kernel uses CUBIC (with Reno as a fallback).

Each side of a connection runs three threads: a listen thread, a ticker thread, and an application thread. While UTCP can support multiple simultaneous fully-duplex connections for our research, we are currently working with two peers—one client and one server—where the server sends data to the client. The listen thread manages all incoming segments for a peer and demuxes them to the intended receiver. It handles three types of segments. First, it processes a synchronize (SYN) request, which initializes a 3WHS between the sender and receiver. The listen thread creates a new TCB using the four-tuple in the segment's header, populates it with the initial connection's values, and adds the TCB to the SYN queue. The SYN queue holds all TCBs that are in the process of establishing a connection. Finally, it sends a SYN-ACK segment to continue the 3WHS. Second, it handles an ACK segment sent in response to a SYN-ACK, which completes the 3WHS. The listen thread then moves the now-established connection's TCB into the accept queue, where an application can call `utcp_accept()` to get the connection's UDP socket descriptor and pop the TCB from the accept queue. Third, it demuxes received segments for an established connection to the correct UTCP socket to be handled accordingly, whether that be a segment with a data payload that will be placed into a receive (RX) buffer, or an ACK for data that was sent out. The ticker thread's only purpose is to wake and decrease a retransmission timeout (RTO) timer's count by one every ten milliseconds. When the timer expires, a handler function is called to perform exponential backoff[3]. The application thread simulates a peer's application functions. For both the client and the server, the thread binds a UDP port to a socket, then a UTCP port to a socket. The rest of its operations differ slightly between the client and the server after the sockets are bound. For the client, the thread calls `utcp_connect()` to initiate a 3WHS with the server. It then allocates an application receive buffer and prepares to receive data from the server. For the server, the thread calls `utcp_accept()` to accept the client's incoming connection request. Then, it allocates an application send buffer and begins sending a stream of bytes to the client.

[3] See RFC 6298, Rule 5.5.

Several of TCP's features have been implemented into UTCP, including:

- Ensuring that data is sent as a reliable in-order stream of bytes.
- A full-duplex connection between peers.
- A three-way handshake process to establish a connection between two peers.
- A SYN and accept queue for tracking and managing connections that are in the process of being established.
- A sliding window for flow control.
- Layer 1 (Application[4]) send and receive buffers, and Layer 2 (Transport) transmit and receive buffers.
- The Window Scale Option in SYN headers to increase the rwnd past the standard 65,535 bytes.
- The Timestamps Option for easy and accurate round-trip time (RTT) calculations.
- Calculating a peer's RTO duration via the Jacobson/Karels Algorithm. The RTO determines how many ticks until a peer's retransmission timer runs out and needs to handle a packet timeout.
- A delayed ACK timer.
- TPC Tahoe, Reno, and NewReno for congestion control and congestion avoidance. These algorithms are fully modular and can be easily switched out for one another using the global `CC_ALGO` object-like macro.
- An out-of-order buffer to minimize the amount of retransmitted data upon packet loss.

All UTCP application-side functions are designed to mirror a BSD socket function.

`utcp_sock()`: Binds a UDP socket, spins up the listen and ticker threads, initializes a TCB for a new UTCP socket, and returns the UTCP socket descriptor. The new TCB is in the `CLOSED` state. This is equivalent to the BSD `sock()` function.

[4] Following the Internet Protocol Suite

`utcp_bind(int utcp_fd, sockaddr_in peer)`: Binds a UTCP port, UDP port, and IPv4 address to the TCB with socket descriptor `utcp_fd`. This is equivalent to the BSD `bind()` function.

`utcp_connect(int utcp_fd, sockaddr_in dest_addr)`: Creates a new TCB upon receiving a SYN packet, initializes it using the four-tuple in the packet's header, moves the `SYN-SENT` state, and sends a SYN packet. Equivalent to the BSD `connect()` function.

`utcp_listen(api_t *global, int backlog)`: Announces that the application is ready to receive connection requests. In this function, the SYN and accept queues are initialized, and the application's TCB moves from the `CLOSED` state to the `LISTEN` state. Equivalent to the BSD `listen()` function.

`utcp_accept(api_t *global)`: Called when the application wants to accept a connection request that is sitting in the accept queue. This function returns the connection's UTCP socket descriptor. Equivalent to the BSD `accept()` function.

`utcp_send(int utcp_fd, void *buf, size_t payload_len)`: Called when the application wants to send data to a peer. This function places up to `payload_len` number of bytes into the sending peer's transmit (TX) buffer and attempts to send the data. Equivalent to the BSD `send()` function.

`utcp_recv(int utcp_fd, void *buf, size_t app_buf_len)`: Called when the application wants to receive data from a peer. This function reads up to `app_buf_len` number of bytes from the RX buffer and writes them into the application's receive buffer. Equivalent to the BSD `recv()` function.

To test UTCP, I created a 1GB text file with random, printable characters and sent it from my personal MacBook Pro (2024 M4 Pro, 24GB RAM) to a Standard B2ats v2 virtual machine hosted by Microsoft Azure in the Central Canada region, over Wi-Fi. The VM acted as a server application, while the MacBook served as a client application. The figures show four to five events, depending on the congestion algorithm: cwnd, ssthresh, triple ACKs, timeouts, and fast recovery (NewReno only). According to RFC 5681, section

3.1, the "initial value of ssthresh SHOULD be set arbitrarily high." As such, the plots have been truncated to the first timeout or triple ACK event that occurs.

TCP Tahoe is the simplest congestion control algorithm of the three. In the event of a triple ACK, Tahoe sets the `ssthresh` to $\text{ssthresh} = \max\left(\frac{1}{2} \cdot \text{bytes in flight},\ 2 \cdot \text{MSS}\right)$, drops the `cwnd` to 1 MSS (maximum segment size), and performs a Fast Retransmit by attempting to resend the dropped packet. In the event that the retransmission timer expires, Tahoe sets the `ssthresh` to $\text{ssthresh} = \max\left(\frac{1}{2} \cdot \text{bytes in flight},\ 2 \cdot \text{MSS}\right)$, drops the `cwnd` to 1 MSS, and resends the oldest unACKed segment. Both events cause the transmission state to reenter Slow Start.

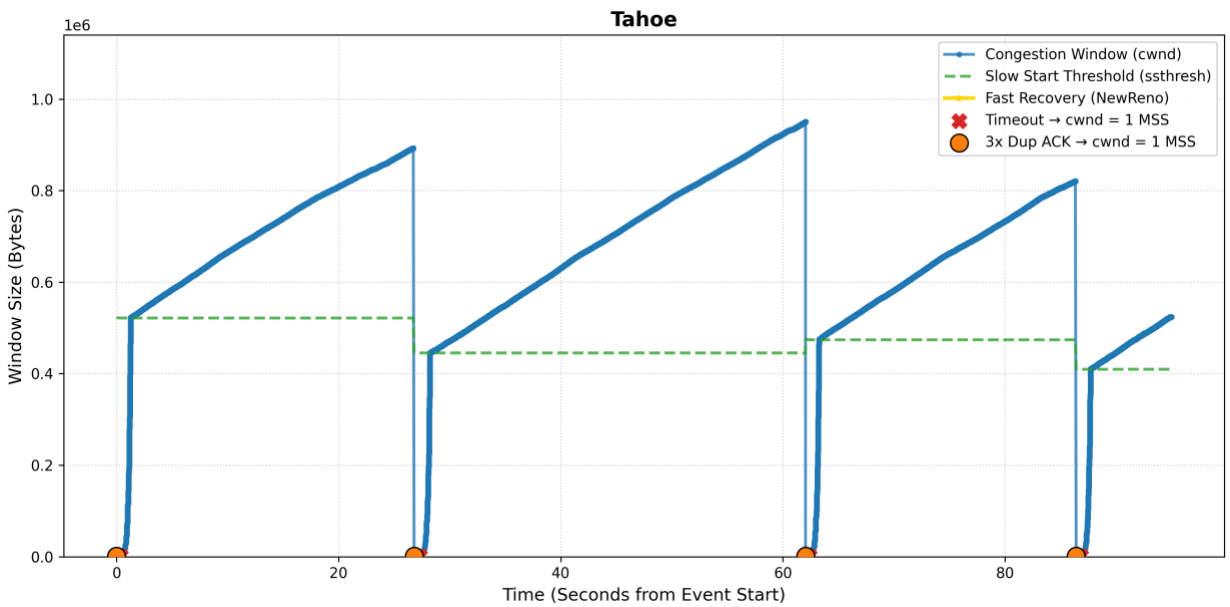

Figure 1 Tahoe congestion window. Total transmission time: 2m1.240s (user 0m6.716s, sys 0m13.280s) Events Count: Triple ACKs=4, Timeouts=4

Reno manages triple ACKs and timeouts more effectively than Tahoe. When a triple ACK occurs, Reno sets the `ssthresh` to $\text{ssthresh} = \max\left(\frac{1}{2} \cdot \text{bytes in flight},\ 2 \cdot \text{MSS}\right)$ (same as Tahoe), and performs a Fast Retransmit. It then sets the `cwnd` to the new `ssthresh` value plus 3 MSS ($\text{cwnd} = \text{ssthresh} + 3 \cdot \text{MSS}$), and enters Fast Recovery, during which it increases the `cwnd` by 1 MSS for each subsequent duplicate

ACK. When a new non-duplicate ACK is received, Reno drops the `cwnd` down to the `ssthresh`, skips Slow Start, and immediately reenters Congestion Avoidance.

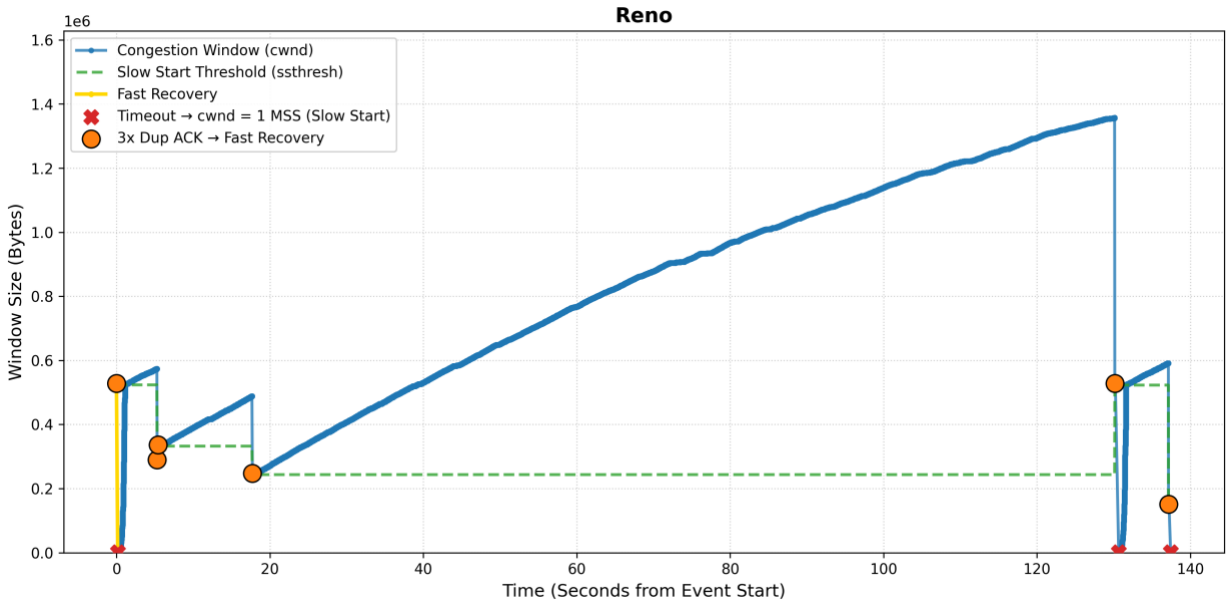

Figure 2 Reno congestion window. Total transmission time: 2m37.264s (user 0m6.896s, sys 0m14.393s) Events Count: Triple ACKs=6, Timeouts=3

A significant drawback of Reno is that when it enters Fast Recovery, the algorithm assumes only one packet was lost during the transmission window. It retransmits the first dropped packet and waits for the packet's ACK. This can be problematic when multiple packets are dropped in the same window because the receiver's ACK will only acknowledge the stream of bytes up to the next lost packet, leaving the other dropped packets unacknowledged (this is called a partial ACK). Once Reno receives the ACK for the first dropped packet, it assumes the recovery period is complete and exits Fast Recovery, leaving both sender and receiver in a standstill. The receiver waits for the dropped packets after the one it just ACKed, and the sender waits to receive ACKs for the data it believes it has successfully sent. This causes Reno to be unaware of the other dropped packets until the retransmission timer expires, resulting in a significant drop in throughput as the sender must reenter Slow Start.

NewReno is an improved version of Reno that correctly handles partial ACKs. The algorithm uses a `recovery` variable to track the highest sequence number that was sent before a packet was lost. While

transmission is in the Fast Recovery stage, when NewReno receives a partial ACK, it assumes that the next packet was also lost, performs a Fast Retransmit for that packet, and stays in Fast Recovery. Only when the sender receives an ACK whose value is greater than or equal to `recovery` does NewReno exit Fast Recovery and reenter Congestion Avoidance.

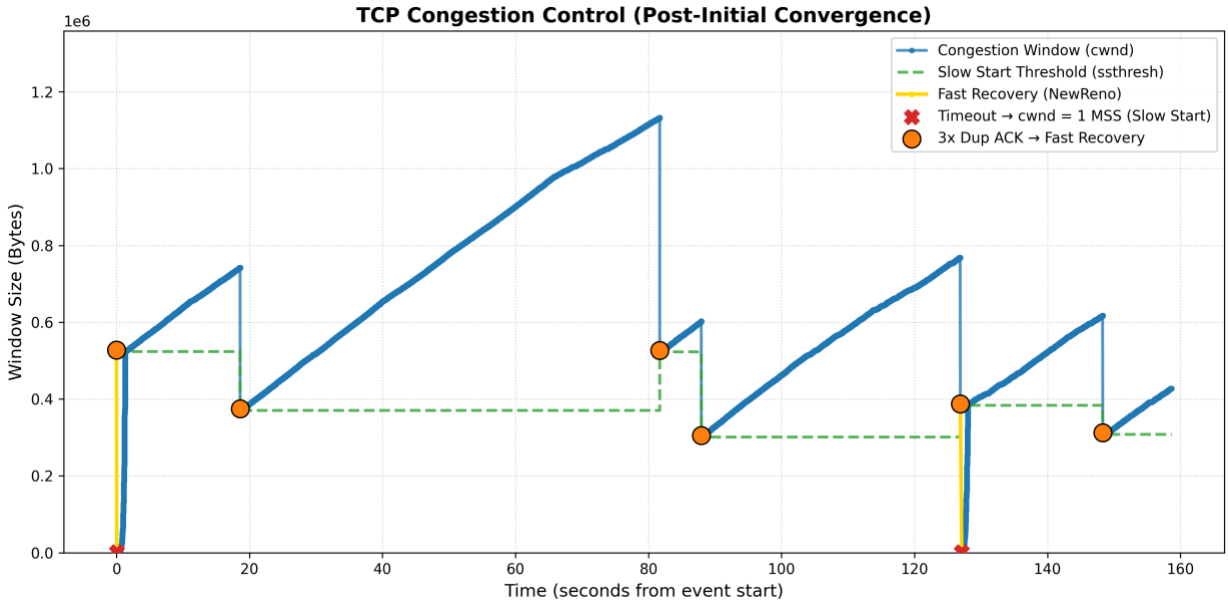

Figure 3 NewReno congestion window. Total transmission time: 2m47.433s (user 0m6.876s, sys 0m13.991s) Events Count: Triple ACKs=6, Partial ACKs=2, Timeouts=2

LSTM Congestion Anomaly Detection

Long Short-Term Memory (LSTM) networks are a type of recurrent neural network (RNN) designed to solve the vanishing gradient problem, an issue that standard RNNs have in capturing long-term dependencies. An LSTM maintains a memory cell, which is controlled by three gates. The memory cell allows the LSTM to select which information to retain or discard, based on how relevant it deems the information, helping the network remember older information for longer periods of time.

My goal was to train an LSTM that could detect when the network was headed towards a congestion event using RTT data. I wanted one of the training features to be a threshold ratio that uses a formula similar to

the RTO timer's duration formula[5]. The duration of the RTO timer is calculated with $\text{RTO} = \text{SRTT} + K \cdot \text{RTTVAR}$, where SRTT is the smoothed RTT, G is the OS's clock granularity, $K = 4$, and RTTVAR is RTT variance. SRTT is an exponentially weighted moving average formula of the RTT. It smooths out RTT calculations, meaning older RTT values are weighted less, which gives a more accurate general idea of what the RTT looks like. RTTVAR is an estimate of how much a subsequent RTT value (R') deviates from the SRTT. ($K \cdot \text{RTTVAR}$) is added to the SRTT for the duration of the RTO as a safety net for the timer. This allows unstable networks to have a longer RTO duration, while more stable networks have a shorter, less generous timer.

My idea was to use a threshold ratio comparing the current RTT to the current RTO. The closer the ratio is to 1, the closer the network is to timing out due to congestion. The threshold is computed as:

$$\boxed{\text{threshold_ratio} = \frac{\text{RTT}}{\text{SRTT} + K \cdot \text{RTTVAR}}}$$

In the equation, all variables are the same as in the RTO formula, with the only difference being that $K = 2$. I used a smaller K value in the formula because I wanted the threshold to indicate that the network has entered a "danger zone" that could lead to a congestion event, rather than only finding RTT values during the event, where it would be too late to prevent it. In practice, the threshold ratio feature was one of seven used in total.

The LSTM was trained on data from 252 runs in which the local client sent a 1GB text file of random characters to the Azure VM, using NewReno as the congestion control algorithm. Due to time constraints, I was unable to collect data on runs using Reno or Tahoe. Each time the client received a packet during a run, the packet's data was logged to a CSV file, which contains:

- Client connection data, such as `cwnd`, `ssthresh`, and the number of newly ACKed bytes.
- RTT data, including `srtt` (smoothed RTT) and `rttvar` (RTT variance).

[5] Defined in RFC 6298.

- Client congestion data, including the client's congestion state (`OPEN`, `RECOVERY`, or `LOSS`), the number of duplicate ACKs, whether the packet is a triple ACK, and whether a timeout has occurred.

First, each CSV file is loaded and normalized by the median of the run's minimum RTT value. The run's utilization, a feature used for training and validation, is then computed by:

$$\boxed{\text{utilization} = \frac{\text{bytes in flight}}{\min(\text{cwnd},\ \text{snd_wnd})}}$$

This value indicates how full the client's congestion window is for a given row. Next, the normalized data is aggregated into 15ms bins. All rows that fall within a bin are collapsed into a single row, either by their average value (e.g., SRTT) or by the maximum value (e.g., queue delay). If a run returns fewer than 201 bins, it is discarded because the LSTM's input size is 200 bins. The threshold ratio is computed for each bin, and any columns with extreme outliers ($-10 < x < 10$, where x is a column's value) are removed.

The data is then labeled by congestion events. A bin that contains a congestion event is labeled as such, and the number of bins just before the congestion bin to be labeled as positive is computed as:

$$\boxed{\text{n_bins} = \max\left(10,\ \frac{\text{SRTT}_{\text{pre}}}{15\ \text{ms}} \cdot 2\right)}$$

This scales the SRTT of the path leading to the congestion event so that ~2 RTTs of history are labeled as positive. When a contiguous set of bins that contains a congestion event occurs, only the first bin is labeled as containing the congestion.

The model was trained via Google Colab on an Nvidia A100 GPU with 80GB of VRAM. It was trained as a stacked LSTM with two recurrent layers for binary classification. The dataset was split 60/20/20 for training, validation, and testing, with the split applied at the run level so that all the windows for a given run only appear once. The input tensor's shape is (`BATCH_SIZE=128, BIN_COUNT=200, N_FEATURES=7`). Each batch contains 128 windows that are processed in parallel, where each window consists of 200 consecutive 15ms bins, and each bin stores the 7 input features to train on. The first LSTM

layer has 128 units (a 128-dimensional hidden state). It reads each window from the input one timestep at a time, then passes its full output sequence to the second layer. The second layer has 64 units and reduces each window's sequence down to a single 64-dimensional output vector. The output is passed to a dense head, which maps the output to a probability between 0 and 1, which represents the likelihood that the next bin is part of a pre-onset congestion window. In total, there were 121,153 trainable parameters. The model was set to train up to 50 epochs, with an early-stopping patience of 8 based on a Precision-Recall Area Under the Curve (PR-AUC) value. It stopped early after epoch 19, and epoch 11's weights were restored because they produced the best results.

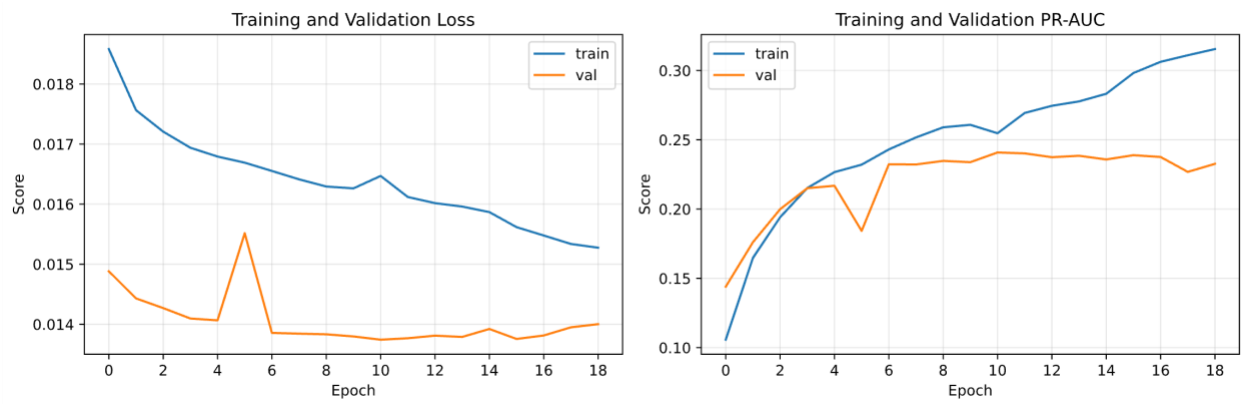

Figure 4 The graph on the left shows the LSTM's training and validation loss up to epoch 19 (the x-axis is 0-18 because it is zero-indexed), where early stopping was activated with a patience of 8. The graph on the right shows its training and validation PR-AUC.

Loss measures the model's performance over time. The training loss curve shows how well the model performs on training data, and the validation loss curve shows how well the model performs on validation data, which is a dataset it has never seen before. The training loss shows a downward trend for most of the learning period, while the validation loss plateaued after epoch 6. There was an unusual spike in validation loss at epoch 5. It is hard to tell what caused the validation loss curve to spike at epoch 5 and then plateau from epoch 6 onward, as many factors could have played a role (e.g., the learning rate may have taken a bad step that temporarily pushed the weights into a worse region before recovering). The gap between the

two curves at epoch 11 (the best epoch) is quite significant, and the validation loss is lower than the training loss throughout.

Precision measures how accurately the model identifies whether a window contains pre-onset congestion bins. Recall measures the percentage of windows the model predicted as positive for pre-onset congestion events, relative to the actual number of pre-onset congestion windows. PR-AUC is a metric that combines the model's precision and recall into a single curve. It measures how well the model ranks positive examples above negative ones. The training PR-AUC curve follows a somewhat mirrored path to the training loss curve. It shows a healthy upward trend throughout the training period, with no signs of plateauing. The validation curve shows an unusual downward spike at epoch 5, then plateaus from epoch 6 onward. At epoch 3, the curves are nearly identical (at ~0.22), but a gap quickly forms between them, and by epoch 18, there is a ~0.10-point (10%) difference between the two. This is a sign that overfitting likely occurred, which is why early stopping was triggered.

I believe the model's poor validation performance is primarily due to insufficient training data, which limited its ability to generalize. The validation curves plateaued while the training curves continued to fluctuate, indicating the model was only memorizing data after epoch 11 rather than learning to generalize.

After the model was trained, it was evaluated on the test set (the remaining 20% of the data). The model achieved PR-AUC $= 0.237$ (23.7%) and ROC-AUC $= 0.755$ (75.5%). PR-AUC was treated as the primary metric because the positive class (windows with pre-onset congestion) was rare, making ROC-AUC (receiver operating characteristic AUC) optimistic. ROC-AUC rewards correct rejection of abundant negatives, whereas PR-AUC reflects performance on the minority class (windows with pre-onset congestion) directly.

The test PR-AUC score matches the validation PR-AUC score (~0.24 at the peak epoch), indicating the model generalized without overfitting to the validation set.

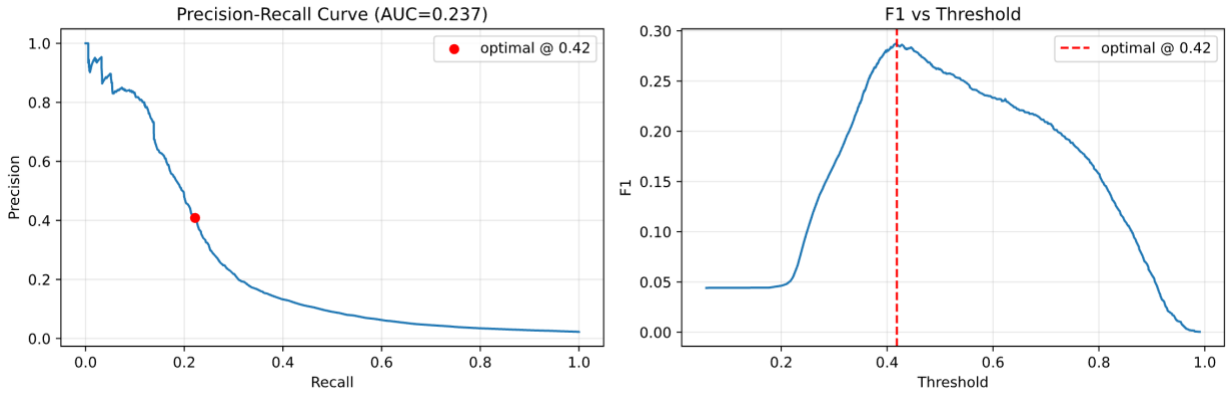

Figure 4 The left graph shows the precision (0.408) and recall (0.222) values of the optimal threshold value. The right graph shows the F1 score (0.288) at the optimal threshold value.

The most optimal threshold value was determined with `precision_recall_curve()`. The function finds the threshold that returns the highest F1 score. The F1 score is the harmonic mean of precision and recall at a single threshold. This metric is used to evaluate a classification model's performance. For my model, the most optimal threshold value was 0.42.

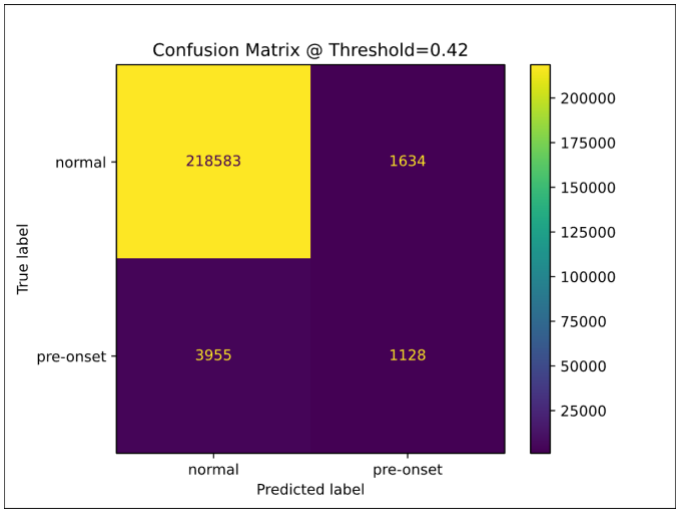

Figure 5 A confusion matrix for the model with a threshold value of 0.42.

The model was re-evaluated on the test set using the optimal threshold. The test set was highly imbalanced, with 220,217 normal windows and 5,083 pre-onset congestion windows (~2.3%). The confusion matrix shows 1,128 true positives, 1,634 false positives, 3,955 false negatives, and 218,533 true negatives. The model achieved $\text{precision} = 0.41$, $\text{recall} = 0.22$, and $\text{F1} = 0.29$ for classifying windows with pre-onset congestion. False positives occurred on ~0.74% of normal windows, indicating the model rarely misflags a non-pre-onset window.

The random-classifier baseline for the test is ~0.23% (equal to the positive class's prevalence). The model's score of 0.237 represents an improvement factor of approximately 10, indicating that the LSTM was able to identify RTT features that suggest a congestion event is imminent, to some degree. It correctly identified 99.2% of normal windows and 22% of pre-onset congestion windows. This indicates that the model is too conservative; it will label a window as pre-onset congestion only if there is very strong evidence that congestion may occur. For every 100 pre-onset windows, the model correctly identified 22, and for every 100 windows it flags as pre-onset congestion, 41 are true pre-onset events.

There is limited research on the use of machine learning for congestion detection and avoidance. I think this is largely due to the inherent difficulty of labeling congestion events in data. Congestion may occur without any detectable onset changes in RTT data, such as a sudden network loss, and network reliability can fluctuate widely between different networks. Furthermore, a network's internal reliability can also range from very reliable to very unreliable due to factors such as congestion at the internet service provider (ISP) level, the amount of traffic going in and out of the network on a given day, and more. That said, there are some areas where my methodology could improve.

The most significant change would be the data collection method. Because most runs were conducted on the same Wi-Fi network, there is little variability in network conditions. A better approach would be to use a tool like tcpreplay to capture PCAP files, or to use a pre-made collection (e.g., Wireshark has a large database of sample captures). This would provide more data for training and significantly diversify the dataset, which could help close the gap between the training and validation PR-AUC curves.

Dropout is a regularization technique used to prevent overfitting. Increasing the dropout rate to 0.4 or 0.5 and adding weight decay to the AdamW optimizer might help narrow the gap between the training and validation loss curves. This would penalize the model for learning overly specific patterns, making the training loss harder to minimize. Training a smaller LSTM model may also help with generalization, because it wouldn't be able to memorize as many training-specific patterns. However, this may produce worse results if paired with a larger amount of training data.

Labeling three RTTs of bins before each congestion onset as positive may help because it gives the model more positive examples per onset. A soft label that decays with distance from the onset bin could be used to teach the model a graded notion of imminence rather than a binary one. This would also help create a separation between triple ACK onsets and timeout onsets.